Chapter 3:
Process Management

Important Terminologies

- Task
  - When a program is requested for execution it is called a task.
    - Before pressing enter key.
- Job
  - When the OS checks the validity of the task by searching it in internal page table, if present then it is said to be a valid job for further processing.
- Process
  - The job is then loaded into computer memory, after memory allocation, the program is set into execution.
  - For example when we write following on the console,
    - C:\>dir
    - A task has been given to OS
    - OS checks its validity by searching it into internal page table, if found, then it is valid, become a job
    - When it become a job, OS must load it into memory.
    - Now this program under execution along with its environment is called a Process.

Process Concept

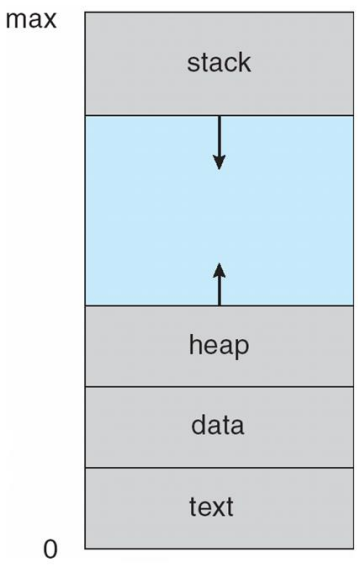
- A process is a program in execution. It is a unit of work within the system. Program is a **passive entity** while a process is an **active entity**.
- One program can be several processes
  - Consider multiple users executing the same program
- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to be executed
  - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some of user/user's, some of operating system that are running concurrently on one or more CPUs
  - Concurrency by multiplexing the CPUs among the processes/threads

Process Management Activities

- The operating system is responsible for the following activities in connection with process management:
  - Creating and deleting both user and system processes
  - Suspending and resuming processes
  - Providing mechanisms for process synchronization
  - Providing mechanisms for process communication
  - Providing mechanisms for deadlock handling

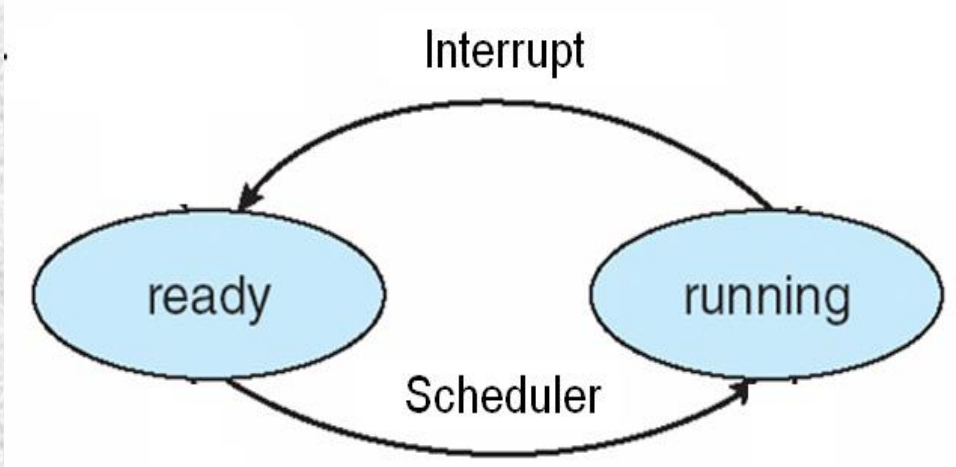
Process in Memory

- A process has multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time



Process State

- As a process executes, it changes the **state**
  - **Ready**: The process is waiting to be assigned to a processor
  - **Running**: Instructions are being executed



Example

Process	Burst Time	Start/Arrival Time
P1	40 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

P1 & P2 States

Process	Burst Time	Start/Arrival Time
P1	40 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

Time Instant	P1	P2
0	Running	Ready
10	Ready	Running
20	Running	Ready
30	Ready	Running
40	Running	Ready
50	Ready	Running
60	Running	Ready
70	-	Running

Class Exercise

Example

Process	Burst Time	Start/Arrival Time
P1	30 sec	0 sec
P2	50 sec	0 sec
CPU time slice = 10 sec		

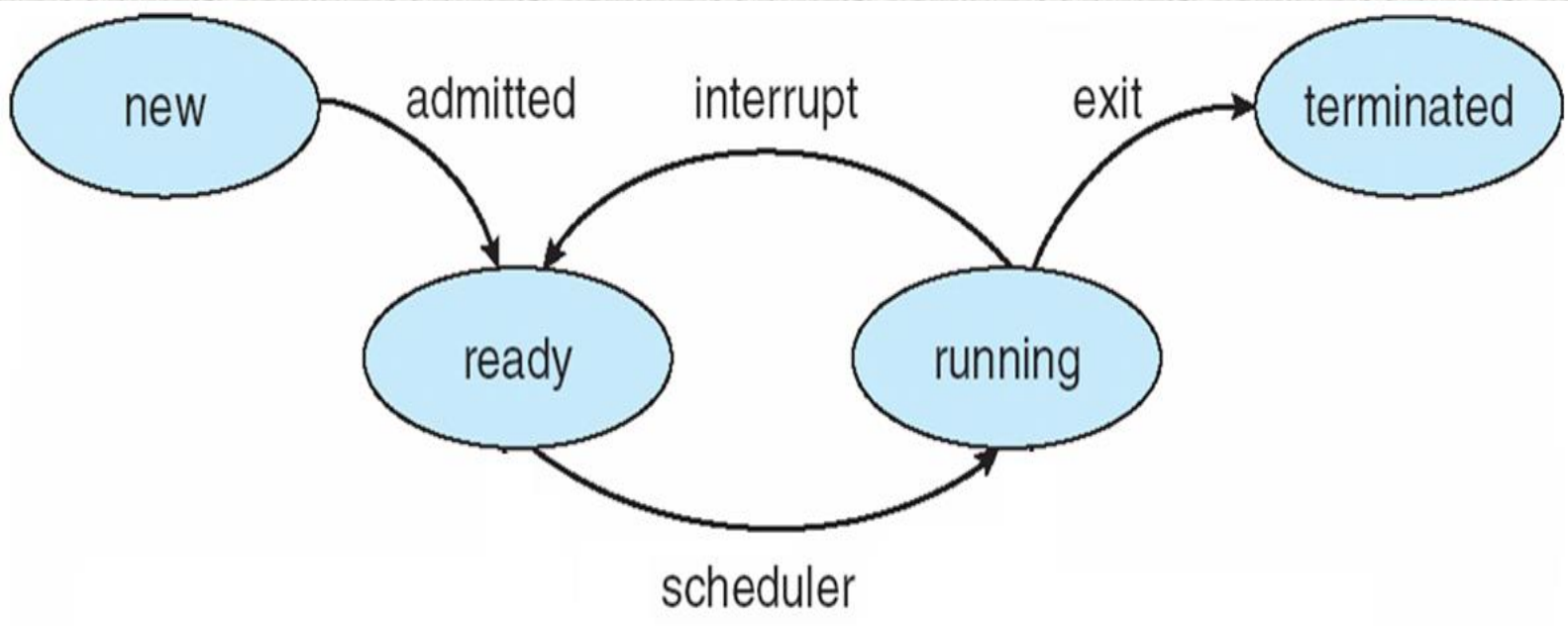
P1 & P2 States

Process	Burst Time	Start/Arrival Time
P1	30 sec	0 sec
P2	50 sec	0 sec
CPU time slice = 10 sec		

Time Instant	P1	P2
0	Running	Ready
10	Ready	Running
20	Running	Ready
30	Ready	Running
40	Running	Ready
50	-	Running
60	-	Running
70	-	Running

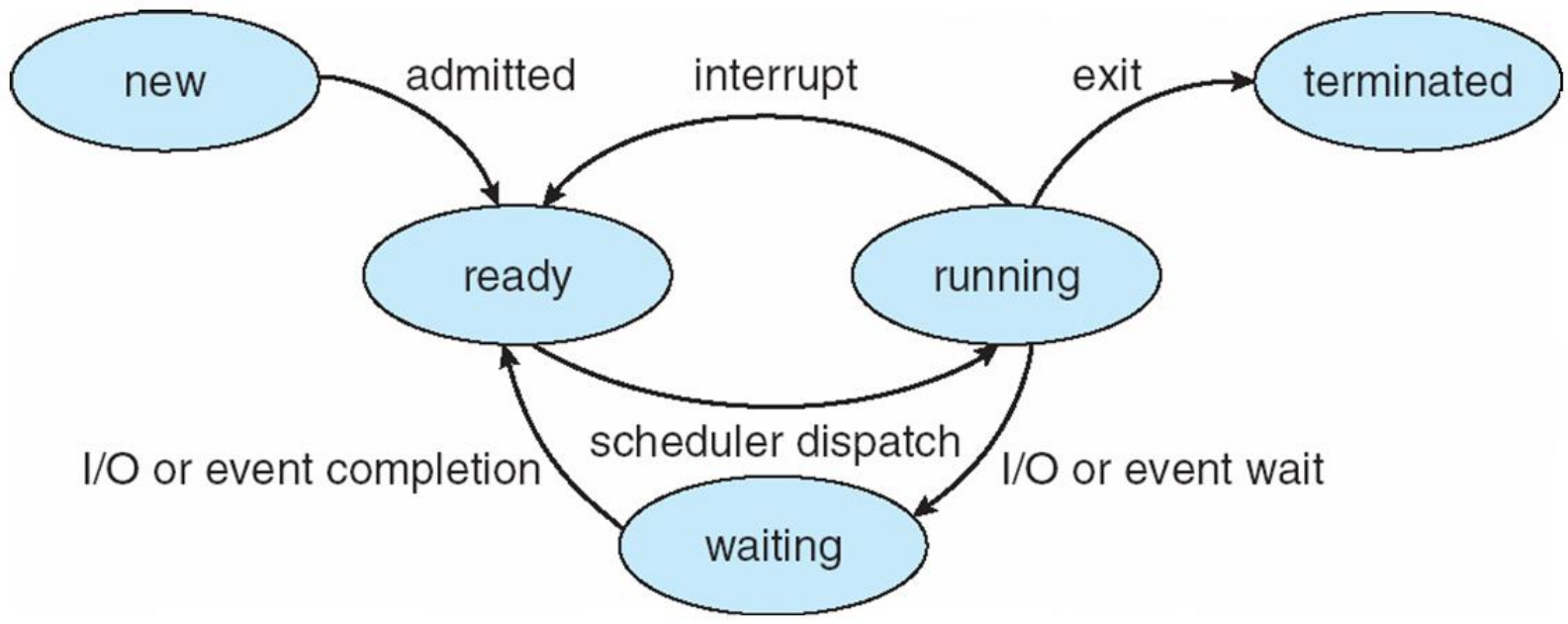
Process State Modified

- As a process executes, it changes **state**
  - **New**: The process is being created
  - **Ready**: The process is waiting to be assigned to a processor
  - **Running**: Instructions are being executed
  - **Terminated**: The process has finished execution



Process State Modified

- As a process executes, it changes **state**
  - **New**: The process is being created
  - **Ready**: The process is waiting to be assigned to a processor
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
  - **Terminated**: The process has finished execution



Example

Process	Burst Time	Start/Arrival Time
P1	40 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

- P1 does an I/O for 10 sec at 20
- P2 does an I/O for 20 sec at 40

P1 & P2 States

Process	Burst Time	Start/Arrival Time
P1	40 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

Time Instant	P1	P2
0	Running	Ready
10	Ready	Running
20	Blocked for I/O	Running
30	Running	Ready
40	Running	Blocked for I/O
50	Running	Blocked for I/O
60	Exit	Running
70	-	Running
		Exit

Class Exercise

Example

Process	Burst Time	Start/Arrival Time
P1	40 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

- P1 does an I/O for (10 x 2) sec at 20 and 50
- P2 does an I/O for (10 x 2) sec at 40 and 60

P1 & P2 States

Process	Burst Time	Start/Arrival Time
P1	40 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

Time Instant	P1	P2
0	Running	Ready
10	Ready	Running
20	Blocked for I/O	Running
30	Running	Ready
40	Running	Blocked for I/O
50	Blocked for I/O	Running
60	Running	Blocked for I/O
70	Exit	Running
		Exit

To make sure… You have Understood !

Process	Burst Time	Start/Arrival Time
P1	30 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

- P1 does an I/O for 10 sec at 20
- P2 does an I/O for 30 sec at 30

P1 & P2 States

Process	Burst Time	Start/Arrival Time
P1	30 sec	0 sec
P2	40 sec	0 sec
CPU time slice = 10 sec		

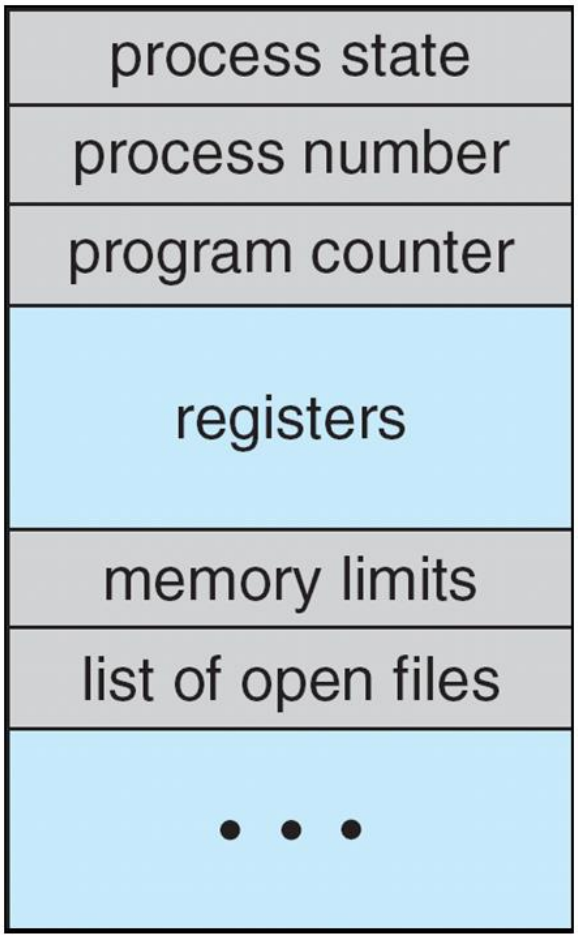
Time Instant	P1	P2
0	Running	Ready
10	Ready	Running
20	Blocked for I/O	Running
30	Running	Blocked for I/O
40	Running	Blocked for I/O
50	Exit	Blocked for I/O
60	-	Running
70	-	Running
		Exit

Process Control Block (PCB)

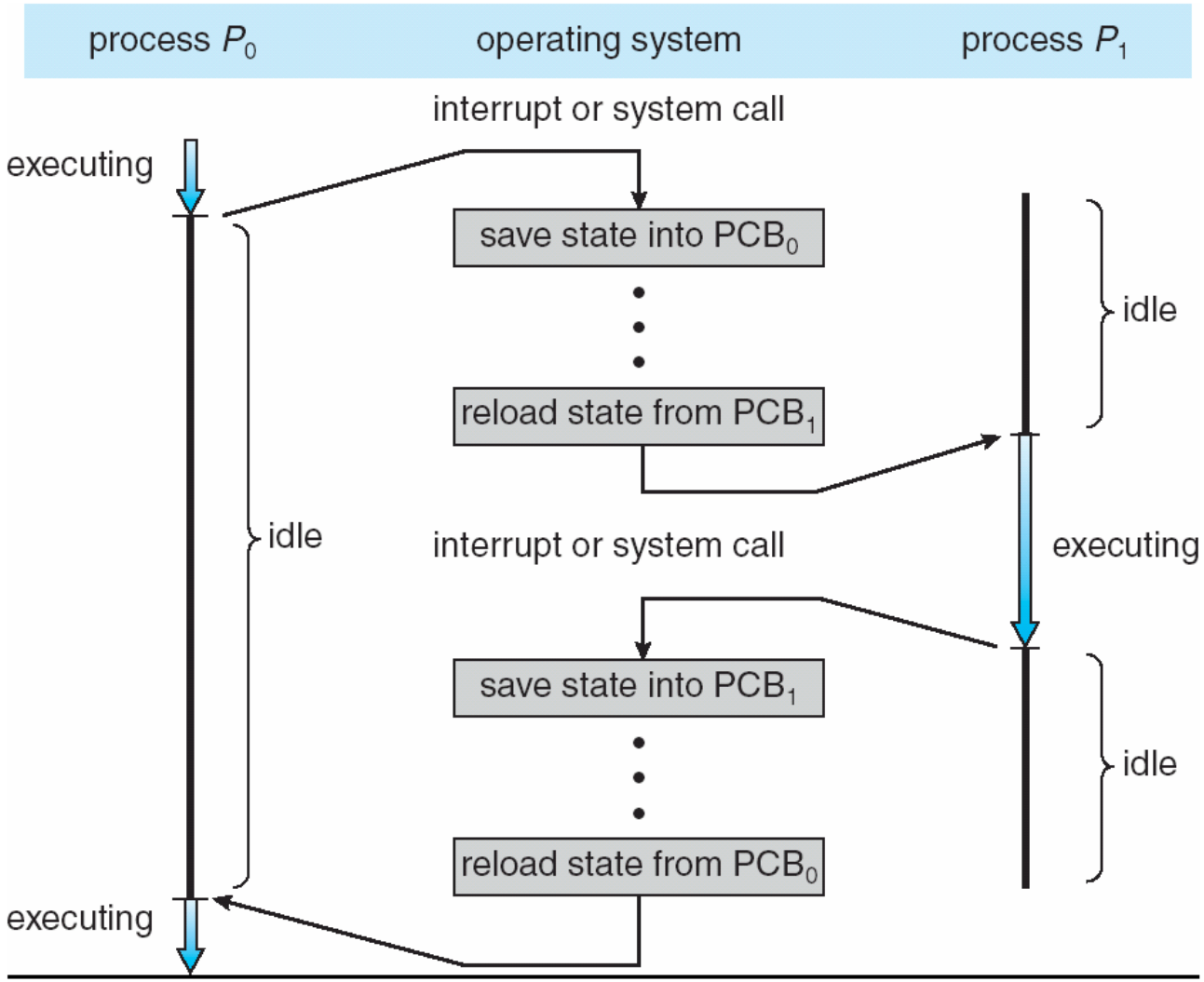
Information associated with each process

(also called **task control block**)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information – priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files



CPU Switch From Process to Process



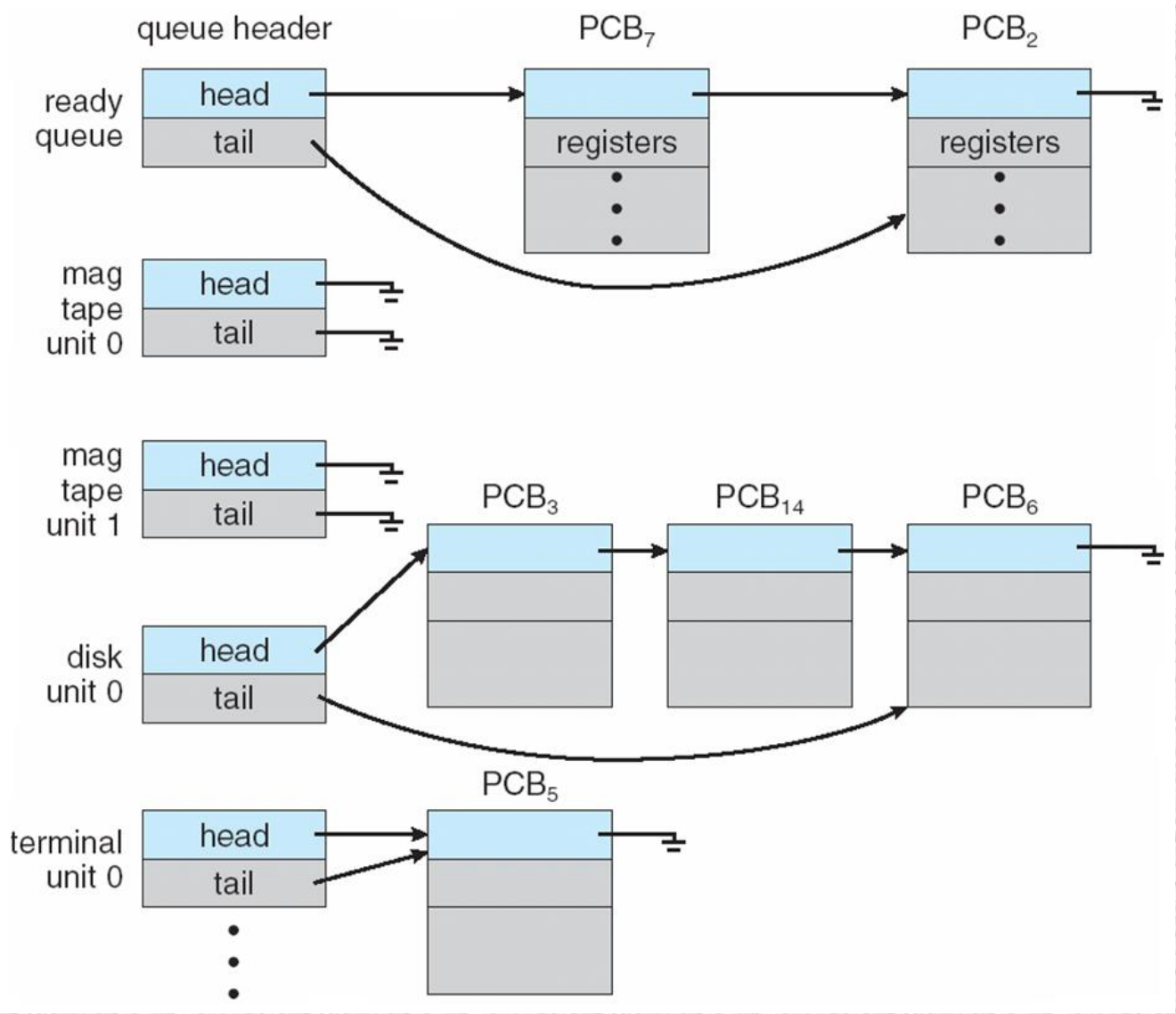
Threads

- So far, process has a single thread of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
    - Multiple threads of control -> **threads**
- Must then have storage for thread details, multiple program counters in PCB.

Process Scheduling

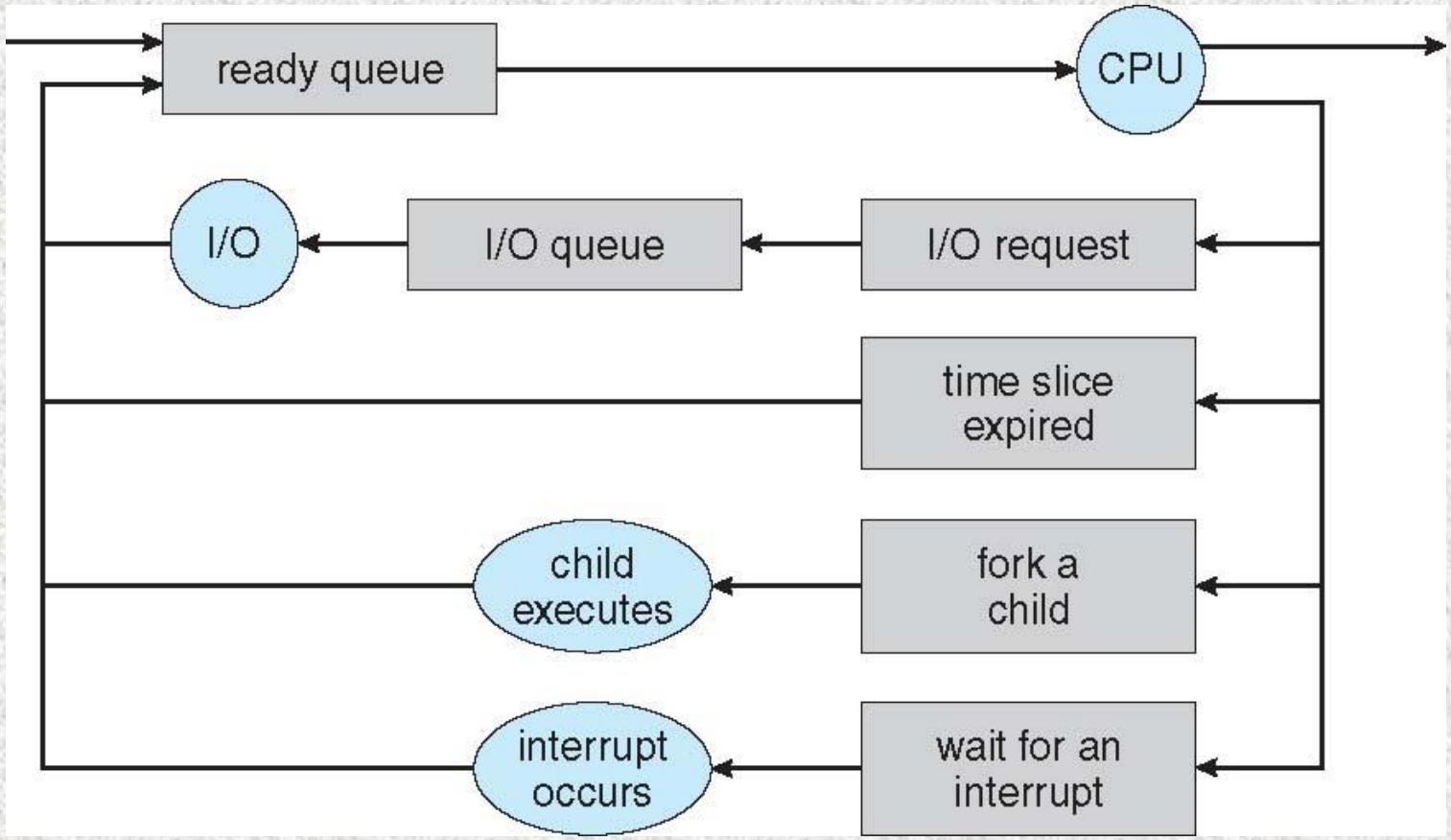
- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues

Ready Queue And Various I/O Device Queues



Representation of Process Scheduling

- **Queueing diagram** represents queues, resources, flows

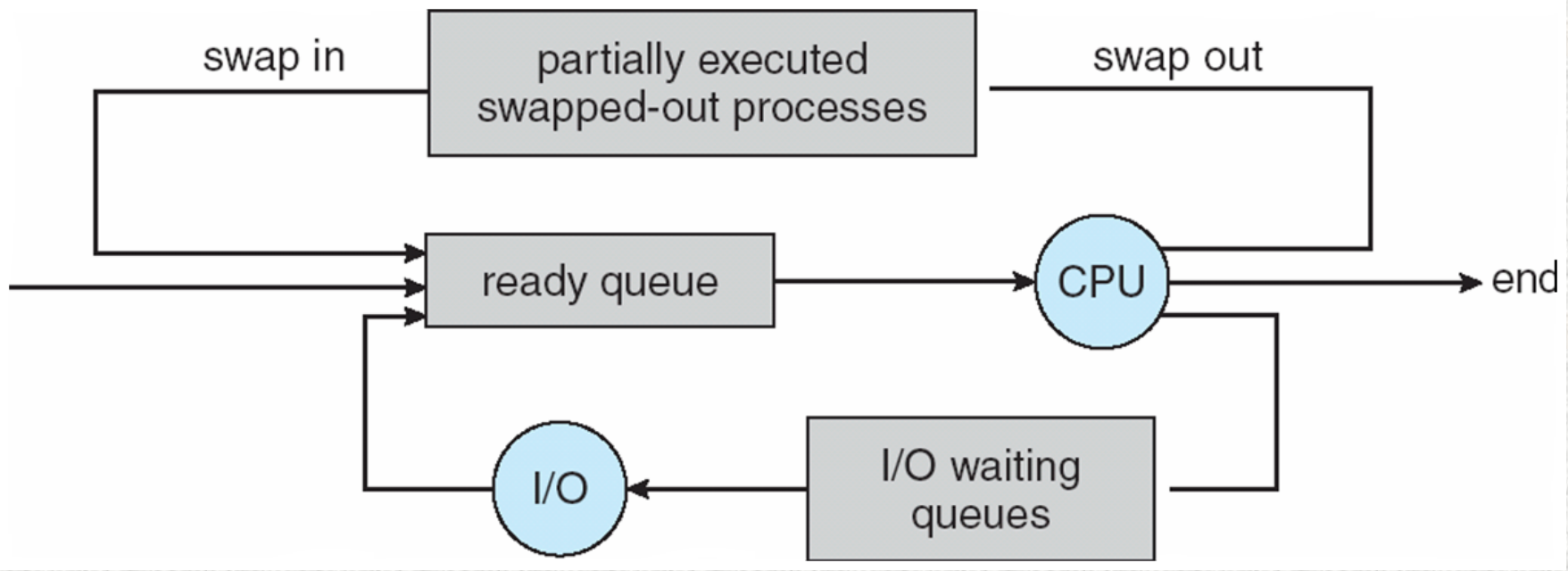


Schedulers

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***

Addition of Medium Term Scheduling

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**



Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

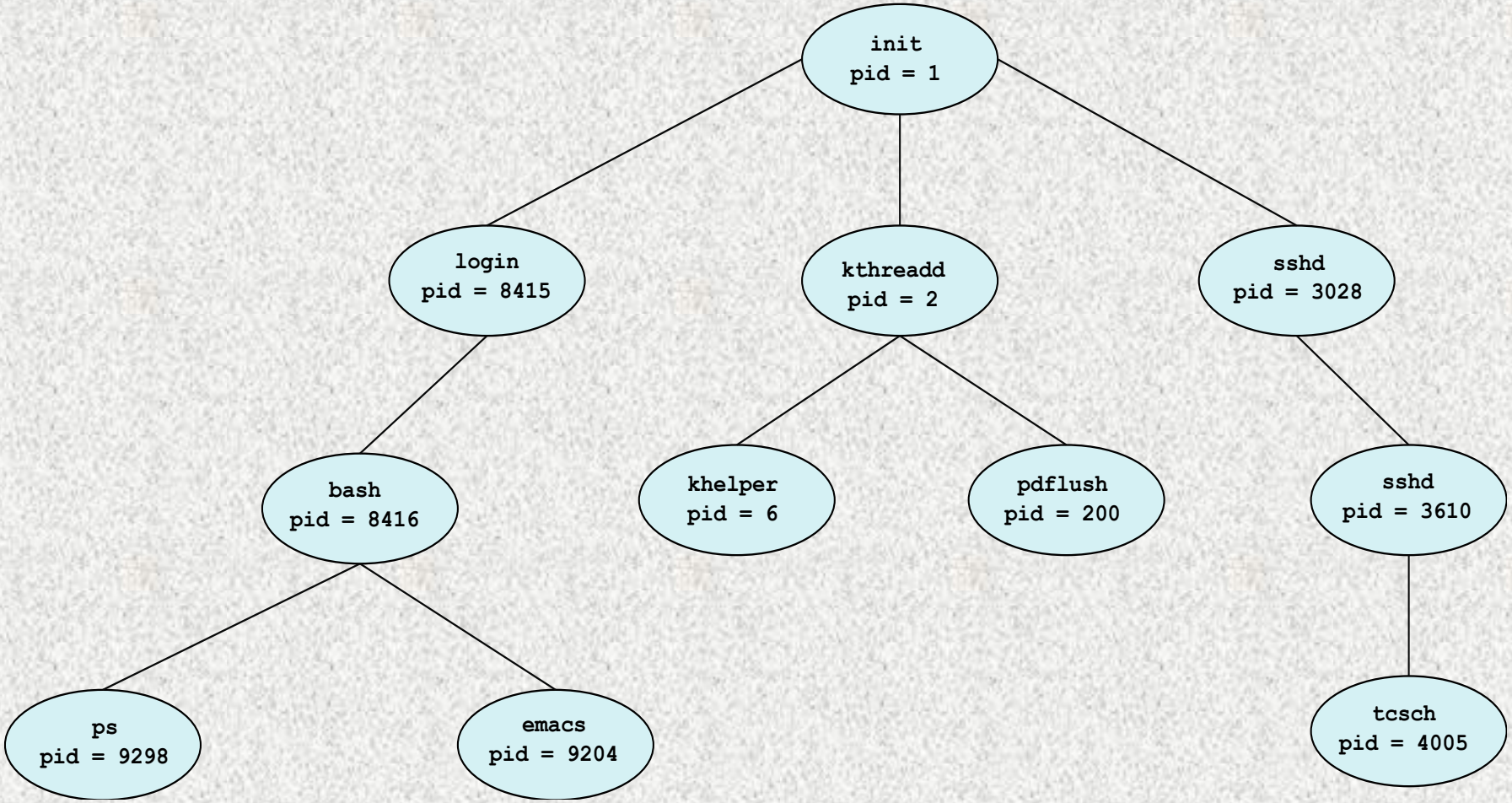
Operations on Processes

- System must provide mechanisms for:
  - process creation,
  - termination,
  - and so on as detailed next.

Process Creation

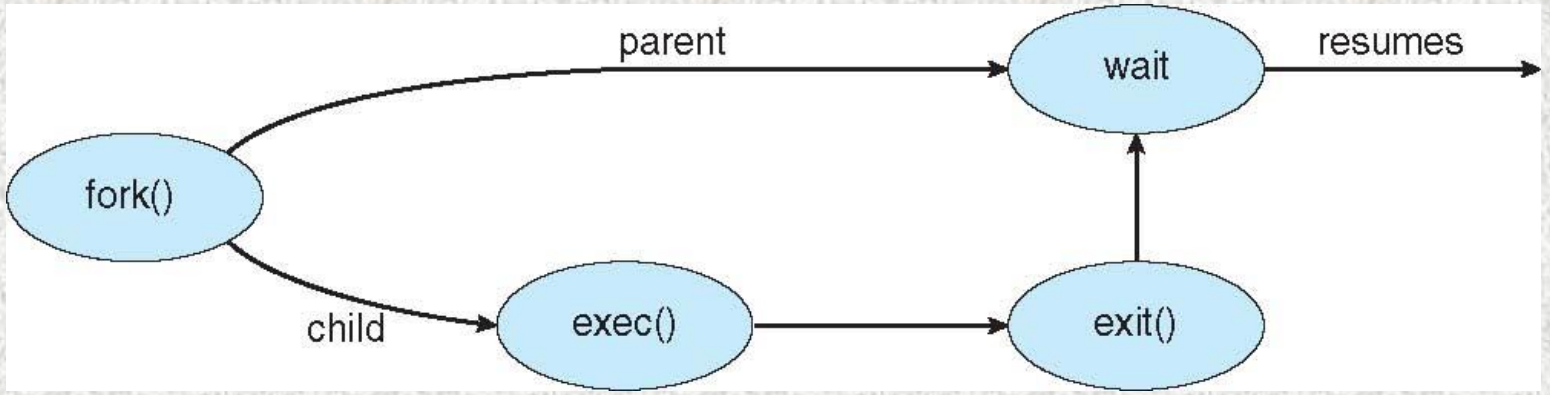
- Parent process create children processes, which in turn create other processes, forming a tree of processes
- Generally, every process is identified and managed via a process identifier (pid)
- Resource sharing options
  - Parent and children share all resources
  - Children can share a subset of parent resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

Example: A Tree of Processes in Linux



Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - For each **`fork()`**, OS creates a copy of process for each node.
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program



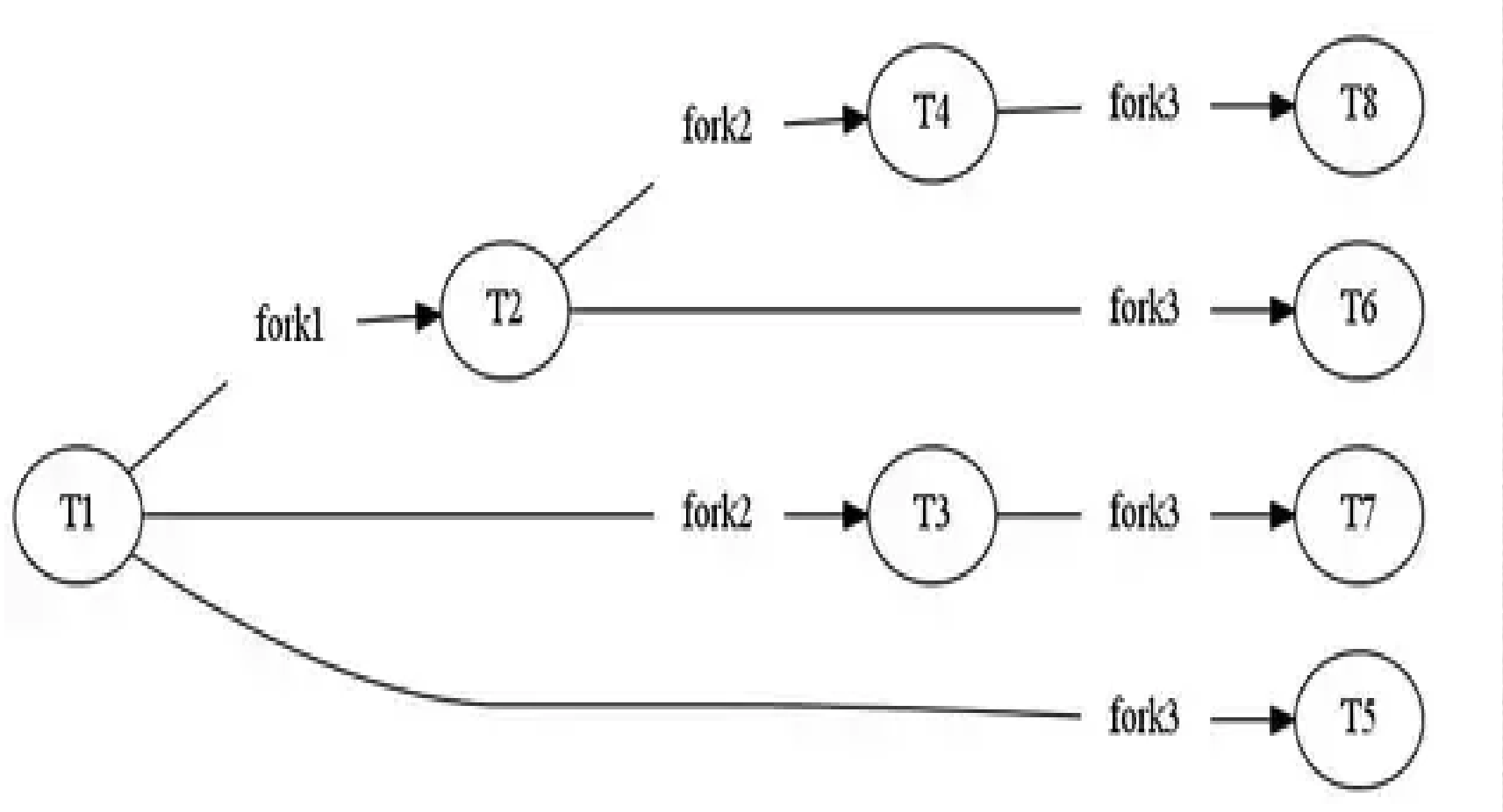
C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Example

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    /* fork a child process */
    fork();

    /* fork another child process */
    fork();

    /* and fork another */
    fork();

    return 0;
}
```

Tree of processes



Process Termination

- Process executes the last statement and asks the operating system to delete it using **exit()** system call.
  - Output data from child to parent (via **wait()**)
  - Process resources are de-allocated by operating system
- Parent may terminate execution of children processes (**abort()**)
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - If parent is exiting
    - Some OS do not allow child to continue if its parent terminates
    - All children terminated - cascading termination
    - The termination is initiated by the operating system.
- Parent may wait for termination of child process using **pid = wait(&status);**
  - If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
  - If parent terminated without invoking **`wait`**, process is an **orphan**